03 System Design

ML System Design Mental Framework (LLM Focused)

A structured approach to ML system design interviews, with emphasis on LLM-specific patterns.

Universal Interview Answer Structure

```
Requirements → Metrics → Pattern Identification → Data → Features / Context
→ Model Choice → Serving Architecture → Scaling → Evaluation → Monitoring
→ Ethics → Tradeoffs
```

Always communicate assumptions and tradeoffs.

---

1. Clarify Requirements

Functional Requirements

Ask:
- What are we building?
- What are the core user actions?
- What is the input and output?
- Is this real-time or batch?

Non-Functional Requirements

Ask:
- **Scale:** How many users? QPS? Data volume?
- **Latency:** What is acceptable? p50? p99?
- **Availability:** What uptime is needed?
- **Cost constraints:** Budget per request? Per user?
- **Privacy / Security:** What data is sensitive? Regulatory requirements?

Summarize Scope

> "Let me confirm my understanding — we are building [X] for [Y] users, with [Z] latency requirement, prioritizing [NFR]."

This shows structured thinking and gives the interviewer a chance to correct assumptions early.

2. Define Success Metrics

Three Levels of Metrics

Business Metrics

What leadership cares about:
- User satisfaction
- Retention
- Engagement
- Acceptance rate (for suggestions)
- Task completion rate

System Metrics

What engineers optimize:
- Latency (p50, p99)
- Throughput (QPS)
- Cost per request
- Availability (uptime %)

Model Metrics

What ML engineers measure:
- Accuracy / Precision / Recall
- **Hallucination Rate** (LLM-specific)
- **Groundedness** (is the answer based on retrieved context?)
- Recall@K (retrieval quality)
- NDCG (ranking quality)

Why All Three Matter

Optimizing only model metrics can hurt business metrics. A 99% accurate model that costs $10 per request and takes 10 seconds is useless if the business needs $0.01 cost and 1-second latency.

3. Identify System Pattern

Pattern A: Pure Generation

Examples: Creative Writing Assistant, Email Writer, Marketing Content Generator

```
User → Prompt Builder → LLM → Response
```

Key considerations:
- Prompt engineering is critical
- No retrieval needed
- Latency dominated by LLM inference
- Hallucination risk is high (no grounding)

Pattern B: RAG (Retrieval-Augmented Generation)

Examples: Enterprise Assistant, Document Q&A, Research Assistant, Legal Assistant

```
User → Retriever → Vector DB → Prompt Builder → LLM → Response
```

Key considerations:
- Retrieval quality directly impacts answer quality
- Chunking strategy matters
- Embedding model choice
- Vector DB selection (HNSW, IVF, brute-force)
- Groundedness is measurable
- Hallucination can be reduced by grounding

Pattern C: Coding Assistant

Examples: GitHub Copilot, Code Review Assistant

```
IDE Context → Code Retrieval → Prompt Builder → LLM → Suggestion
```

Key considerations:
- Context window management (open files, imports, repo structure)
- Latency is critical (suggestions must appear as you type)
- Code-specific retrieval (semantic + syntactic)
- Acceptance rate is the key product metric

Pattern D: Agent

Examples: Research Agent, Travel Planner, Multi-Agent Assistant

```
User → Planner → Tool Calls → LLM → Response
```

Key considerations:
- Tool selection and orchestration
- Multi-step reasoning
- Error handling for tool failures
- Latency is high (multiple LLM calls)

- Cost per request is high
- Evaluation is hard (multi-step correctness)

Pattern E: Recommendation / Ranking

Examples: YouTube, Netflix, Product Recommendations

```
User → Candidate Generation → Ranking Model → Results
```

Key considerations:
- Two-stage: candidate generation (fast, broad) → ranking (slower, precise)
- Feature engineering is critical
- Recall@K and NDCG are key metrics
- Cold start problem for new users/items
- Online learning / model freshness

Pattern F: Classification / Prediction

Examples: Fraud Detection, Churn Prediction, Spam Detection

```
Features → Model → Prediction
```

Key considerations:
- Feature pipeline is the bulk of the work
- Model interpretability may matter (regulatory)
- Real-time scoring vs batch scoring
- Class imbalance handling
- Precision-recall tradeoff (cost of false positive vs false negative)

---

4. Data Ingestion & Preprocessing

Questions to Ask

- Where does data come from?
- How is it collected?
- How is it cleaned?
- Any augmentation needed?
- Any filtering needed?

Common Data Sources

Pattern	Data Sources
RAG	Documents, PDFs, wikis, knowledge bases
Coding Assistant	Repositories, code files, documentation
Recommendation	User interactions, clicks, views, purchases
Classification	Logs, user events, transaction records
Agent	Tool outputs, API responses, web pages

Preprocessing Steps

1. **Collection:** Batch import, streaming ingestion, API polling
2. **Cleaning:** Remove duplicates, fix encoding, handle missing data
3. **Chunking** (for RAG): Split documents into retrievable units
4. **Embedding:** Convert text/code into vector representations
5. **Indexing:** Store vectors in vector DB for fast retrieval

---

5. Features / Context

The Key Question

> "What information does the model need to do its job?"

Context by Pattern

Pattern	Context Needed
ChatGPT	Conversation history, memory, system prompt, retrieved documents
Copilot	Current file, imports, open tabs, repo structure, cursor position
Creative Writing	Genre, tone, audience, style examples
RAG	Retrieved chunks, metadata, user query, conversation history
Agent	Tool descriptions, previous tool outputs, user goal, plan state
Recommendation	User features, item features, interaction history, context (time, device)

Context Window Management

For LLM systems, context window is a scarce resource:

- **Prioritize:** Most relevant context first
- **Truncate:** Drop old conversation turns
- **Compress:** Summarize long documents before including
- **Rerank:** Use a smaller model to rerank retrieved chunks before feeding to LLM

---

6. Model Selection

Key Questions

- LLM or traditional ML?
- Fine-tuning needed?
- RAG needed?
- Tool use needed?
- Small model vs large model?

The Quality-Latency-Cost Triangle

You can optimize for at most two of three.

Decision Framework

Need	Model Choice
High quality, cost OK	Large model (GPT-4, Claude, Gemini Ultra)
Balanced	Medium model (Llama 70B, Mistral)
Low latency, low cost	Small model (Llama 8B, Gemma 2B)
Domain-specific accuracy	Fine-tuned smaller model
Grounded answers	RAG + any model

Need	Model Choice
Multi-step reasoning	Large model + tool use

Fine-Tuning vs RAG vs Prompt Engineering

Approach	When to Use	Cost	Latency Impact
Prompt Engineering	Quick iteration, general tasks	Low	None
RAG	Knowledge-intensive, dynamic data	Medium	Retrieval latency
Fine-tuning	Domain-specific style/format, consistent behavior	High	None (same inference)
All three	Production-grade domain system	High	Combined

---

7. Serving Architecture

Typical Flow

```
User → API → Rate Limiter → Cache → DB → Build Context → Queue
→ Inference Service → Model → Response → DB → User
```

Common Components

Component	Purpose
Rate Limiter	Prevent abuse, control cost
Cache	Cache common queries / responses
Database	Store user data, conversation history
Queue	Buffer requests during traffic spikes
Load Balancer	Distribute across inference servers
Inference Service	Run model inference (GPU servers)

LLM-Specific Serving Concerns

- **GPU allocation:** Models need GPU memory. Plan for model size + KV cache.

- **Batching:** Group requests for efficient GPU utilization
- **Streaming:** Stream tokens to user for perceived low latency
- **Model versioning:** Blue-green deployments for model updates
- **Fallback:** If large model is overloaded, fall back to smaller model

---

8. Scaling & Optimization

Latency Optimization

Technique	How It Helps
Streaming	User sees first token in ~200ms instead of waiting for full response
Caching	Common queries return instantly from cache
Prompt compression	Shorter prompts = faster inference
Speculative decoding	Small model drafts, large model verifies
KV cache	Avoid recomputing past tokens during generation

Throughput Optimization

Technique	How It Helps
Batching	Process multiple requests in one forward pass
Autoscaling	Add GPU servers during peak, remove during low traffic
Queueing	Buffer requests, process in batches
Continuous batching	Dynamically add/remove requests from active batch

Cost Optimization

Technique	How It Helps
Smaller models	Lower GPU cost per request
Quantization	Reduce model precision (fp16 → int8 → int4)
Caching	Avoid redundant inference
Distillation	Train smaller model to mimic larger model
Route by complexity	Easy queries → small model, hard queries → large model

Reliability

Technique	How It Helps
Fallback models	If primary model fails, use backup
Retries	Retry transient failures with backoff
Circuit breakers	Stop sending traffic to failing service
Graceful degradation	Return cached or simplified response during outages

---

9. Evaluation

Offline Evaluation

Metric	What It Measures
Accuracy	Overall correctness
Precision	Of positive predictions, how many are correct
Recall	Of actual positives, how many did we find
F1	Harmonic mean of precision and recall
Recall@K	Of top-K results, how many relevant items are found
NDCG	Ranking quality (normalized discounted cumulative gain)
BLEU / ROUGE	Text generation quality (n-gram overlap)
Perplexity	Language model quality (lower is better)

Online Evaluation

Metric	What It Measures
CTR	Click-through rate (recommendations)
Acceptance Rate	How often users accept suggestions (coding assistant)
User Satisfaction	Survey scores, thumbs up/down
Retention	Do users come back?
Task Completion	Did the user complete their task?

A/B Testing

Test variations to find the best configuration:
- **Prompt versions:** Different system prompts
- **Model versions:** Different model sizes or fine-tunes
- **Retrieval strategies:** Different chunk sizes, embedding models, reranking

---

10. Monitoring

System Monitoring

Metric	Alert Threshold
Latency (p99)	> target latency
Error rate	> 1% of requests
Throughput	Unexpected drop
GPU utilization	> 90% (scale needed) or < 20% (scale down)

Model Monitoring

Metric	Why It Matters
Hallucinations	Model generating false information
Drift	Input distribution changing over time
Quality	Output quality degrading (user feedback)

Business Monitoring

Metric	Why It Matters
Engagement	Are users using the feature?
Satisfaction	Are users happy with results?
Adoption	Is usage growing?

---

11. Safety / Ethics

Considerations

Concern	Mitigation
Bias	Diverse training data, bias audits, fairness metrics
Privacy	PII detection, data retention policies, user consent
Toxicity	Content filters, safety classifiers, red-teaming
Hallucinations	Grounding (RAG), confidence thresholds, human review
Copyright	Training data provenance, output filtering
Data leakage	Prompt injection defenses, output sanitization

LLM-Specific Safety

- **Prompt injection:** User input tries to override system instructions
- **Jailbreaking:** Users attempt to bypass safety guardrails
- **Data exfiltration:** Model reveals training data or system prompts
- **Model inversion:** Attackers reconstruct training data from outputs

---

Quick Reference: The 11-Step Framework

```
 1. Requirements     → What are we building? For whom? What NFRs?
 2. Metrics          → How do we know it's working? (Business + System + Model)
 3. Pattern          → Which system pattern? (Generation, RAG, Agent, etc.)
 4. Data             → Where does data come from? How is it processed?
 5. Features/Context → What does the model need to see?
 6. Model Choice     → Which model? Fine-tune? RAG? Tradeoffs?
 7. Serving          → How do we serve it? (API, cache, queue, GPU)
 8. Scaling          → How do we handle growth? (Batching, autoscaling)
 9. Evaluation       → How do we measure quality? (Offline + Online)
10. Monitoring       → How do we detect problems? (System + Model + Business)
11. Ethics           → What are the risks? (Bias, privacy, hallucinations)
```

Always end with tradeoffs. Every choice has a cost. State it explicitly.

---

12. LLM Serving Architecture Deep Dive

GPU Capacity Planning

```
GPU Memory Budget = Model Weights + KV Cache + Activations + Overhead

Example: LLaMA-2 70B on 8x A100 80GB
  Weights (FP16):     140 GB → 17.5 GB/GPU (tensor parallelism)
  KV Cache (batch=32, ctx=4096): ~32 GB → 4 GB/GPU
  Activations:        ~8 GB → 1 GB/GPU
  Overhead:           ~2 GB/GPU
  Total per GPU:      ~24.5 GB (fits in 80 GB)
```

Serving Stack Components

```
User Request
    ↓
API Gateway (auth, rate limiting, routing)
    ↓
Request Queue (Kafka/SQS for async, or in-memory for sync)
    ↓
Scheduler (continuous batching, priority, admission control)
    ↓
Inference Engine (vLLM, TGI, TensorRT-LLM)
    ↓
Model (tensor-parallel across GPUs)
    ↓
Response Stream (SSE/WebSocket for token streaming)
    ↓
User
```

Key Serving Decisions

Decision	Options	Tradeoff
Serving framework	vLLM, TGI, TensorRT-LLM, Triton	Throughput vs flexibility vs vendor lock-in
Parallelism	Tensor parallel (TP), Pipeline parallel (PP), Data parallel (DP)	TP: communication heavy; PP: bubble overhead; DP: simplest
Batching	Static, dynamic, continuous	Static: simple but wasteful; Continuous: optimal GPU util
Quantization	FP16, INT8, INT4	Quality vs memory vs speed
Caching	Prefix cache, semantic cache	Hit rate vs freshness

Model Routing (Cascading)

```
User Query → Classifier → Simple? → Small Model (8B)
                        → Medium? → Medium Model (70B)
                        → Complex? → Large Model (GPT-4)
```

```
def route_request(query, classifier):
    complexity = classifier.score(query)
    if complexity < 0.3:
        return small_model.generate(query)    # 8B, fast, cheap
    elif complexity < 0.7:
        return medium_model.generate(query)   # 70B, balanced
    else:
        return large_model.generate(query)    # GPT-4, slow, expensive
```

Benefits: 70-80% of queries are simple → route to small model → 10x cost savings with minimal quality loss.

Continuous Batching (L5 Critical)

Traditional batching waits for all requests in a batch to finish → GPU idle time. Continuous batching:

```
Time:  t1    t2    t3    t4    t5
Req A: [gen] [gen] [gen] [done]
Req B:       [gen] [gen] [gen] [gen] [done]
Req C:             [gen] [gen] [gen] [done]
Req D:                   [gen] [gen] [gen] [done]

GPU never idle — new requests join as old ones finish
```

This is what vLLM and TGI implement. It's the single biggest throughput improvement for LLM serving.

Streaming Architecture

```
Client ← SSE ← API ← Token Stream ← Inference Engine

Token 1: "The" → sent immediately (~200ms TTFT)
Token 2: "answer" → sent as generated (~20ms TPOT)
Token 3: "is" → sent as generated
...
```

TTFT (Time To First Token): dominated by prefill (processing the prompt).
TPOT (Time Per Output Token): dominated by decode (one forward pass per token).

Multi-Model Serving

```
GPU 0-3: Model A (70B, TP=4)
GPU 4-5: Model B (8B, TP=2)
GPU 6-7: Model C (8B, TP=2) — for fallback/overflow
```

Considerations:
- **Model placement**: which GPUs hold which models.
- **Request routing**: send to the right model's GPU group.
- **Hot-swapping**: load/unload models without downtime.
- **Multi-LoRA**: one base model + multiple adapters (see LoRA notes).

---

13. RAG System Design (End-to-End)

Architecture

```
User Query
    ↓
Query Rewriter (LLM: HyDE, multi-query, step-back)
    ↓
Hybrid Retriever
    ├── BM25 (Elasticsearch) → top 50
    └── Dense (FAISS/HNSW)   → top 50
    ↓
Reciprocal Rank Fusion (RRF) → top 50 merged
    ↓
Cross-Encoder Reranker → top 5
    ↓
Context Builder (chunk selection, token budget management)
    ↓
LLM Generation (with citation instructions)
    ↓
Response + Citations → User
```

Key Design Decisions

Component	Decision	Impact
Embedding model	bge-large-en (1024 dim) vs OpenAI ada (1536 dim)	Quality vs cost
Chunk size	512 tokens, 10% overlap	Precision vs context
Vector DB	FAISS (in-memory) vs Pinecone (managed) vs pgvector (Postgres)	Latency vs ops vs integration

Component	Decision	Impact
Reranker	bge-reranker-large vs Cohere Rerank	Quality vs cost
Index update	Real-time incremental vs nightly batch	Freshness vs complexity

Indexing Pipeline

```
Documents → Parser (PDF/HTML/Markdown) → Semantic Chunker
    ↓
Embedding Model → Vectors
    ↓
Vector DB (HNSW index) + Metadata Store (Postgres)
    ↓
Incremental updates for new/modified documents
Nightly full re-index for deletions
```

Token Budget Management

```
Total context budget: 8192 tokens
  System prompt:     500 tokens
  Retrieved context: 5000 tokens (5 chunks × 1000 tokens)
  Conversation:      1000 tokens
  Response buffer:   1692 tokens

If retrieved context exceeds budget → truncate or summarize
```

---

14. Agent System Design

Architecture

```
User Goal
    ↓
Planner LLM: decompose goal into steps
    ↓
Step 1: Select tool → Execute → Observe result
Step 2: Select tool → Execute → Observe result
...
Step N: Synthesize final answer
    ↓
Response → User
```

Tool Selection

```
Available tools:
  - search_web(query) → search results
  - calculator(expression) → numeric result
  - code_executor(code) → execution output
  - database_query(sql) → query results

Planner prompt: "Given the goal and available tools, which tool should I use next?"
```

Key Challenges

Challenge	Solution
Tool failure	Retry with backoff, fallback to alternative tool
Infinite loops	Max steps limit, cycle detection
Cost control	Budget per request (max LLM calls, max tool calls)
Latency	Parallel tool execution where possible
Evaluation	Step-by-step correctness checking

Multi-Agent Architecture

```
User → Orchestrator Agent
         ├── Research Agent (search + summarize)
         ├── Code Agent (write + execute code)
         └── Critic Agent (verify outputs)
```

Each agent has its own system prompt, tools, and evaluation criteria. The orchestrator coordinates and synthesizes.

---

15. L5 Interview Q&A

Q: "Design an LLM serving system for 10M daily users."

Scale estimation:
- 10M users, ~5 queries/day = 50M queries/day
- ~580 queries/second average, ~3000 QPS peak (5x)
- Average response: 200 tokens, average prompt: 500 tokens

Architecture:
1. **API Gateway**: auth, rate limiting, request routing.

2. **Model selection**: route by complexity — 80% to 8B model, 15% to 70B, 5% to GPT-4.
3. **Serving**: vLLM with continuous batching, tensor parallelism (TP=4 for 70B, TP=1 for 8B).
4. **GPU fleet**: 8B model on 4 GPU groups (1 GPU each), 70B on 4 GPU groups (4 GPUs each).
5. **Caching**: prefix cache for shared system prompts, semantic cache for common queries.
6. **Queue**: Kafka for async requests, in-memory for sync streaming.
7. **Monitoring**: TTFT < 500ms, TPOT < 50ms, GPU utilization 60-80%.

Cost estimate:
- 80% queries on 8B: 40M queries × 700 tokens × $0.0001/1K tokens = $2,800/day
- 15% on 70B: 7.5M × 700 × $0.001/1K = $5,250/day
- 5% on GPT-4: 2.5M × 700 × $0.03/1K = $52,500/day
- Total: ~$60K/day → routing to small models saves ~$50K/day vs all-GPT-4.

Q: "How would you reduce LLM serving costs by 10x?"

1. **Model routing**: 80% of queries to small model → 10x cheaper for those.
2. **Quantization**: INT4 for 70B → 4x less memory → more concurrent requests per GPU.
3. **Caching**: semantic cache for common queries → 20-30% hit rate = 20-30% fewer inference calls.
4. **Speculative decoding**: small model drafts, large model verifies → 2-3x throughput.
5. **Batching**: continuous batching → 3-5x GPU utilization improvement.
6. **Prompt compression**: reduce prompt length → less prefill compute.
7. **Off-peak scaling**: autoscale down during low traffic.

Q: "How do you handle a traffic spike where QPS doubles suddenly?"

1. **Autoscaling**: pre-provisioned GPU pools that spin up in 2-5 minutes.
2. **Queue + backpressure**: buffer excess requests, degrade gracefully (shorter responses, smaller model).
3. **Model fallback**: if 70B is overloaded, route to 8B with a warning.
4. **Rate limiting**: throttle low-priority requests, prioritize paying users.
5. **Circuit breakers**: stop sending traffic to overloaded GPU groups.
6. **Pre-warming**: keep spare GPUs loaded with model weights (ready to serve).

Q: "Design a RAG system that stays fresh as documents are updated."

1. **Incremental indexing**: new/modified documents → embed → add to vector DB in real-time.
2. **Versioning**: each chunk has a version timestamp; retrieval prefers recent versions.
3. **TTL on cached responses**: cached RAG answers expire after 1 hour (or on document update).
4. **Webhook integration**: document management system sends update events → trigger re-index.
5. **Nightly full re-index**: catch deletions and fix any incremental errors.
6. **Stale detection**: if retrieved chunks are >30 days old, flag to user ("Information may be outdated").

Q: "How would you evaluate an LLM system in production?"

Offline:
- Benchmark suite: MMLU, GSM8K, HumanEval, domain-specific eval set.
- RAGAS: faithfulness, answer relevance, context precision/recall.
- Regression testing: ensure new model versions don't break existing capabilities.

Online:
- A/B testing: new model vs current model, measure user satisfaction.
- Human eval: sample 100 responses/day, human raters score quality.
- User feedback: thumbs up/down, detailed feedback forms.
- Business metrics: task completion rate, retention, support ticket deflection.

Monitoring:
- Quality drift: track faithfulness, hallucination rate over time.
- Safety: toxicity classifier, jailbreak detection, prompt injection attempts.
- Performance: TTFT, TPOT, error rate, GPU utilization.

Quick Reference: LLM Serving Cheat Sheet

Problem	Solution
High latency (TTFT)	Prefix caching, smaller model, prompt compression
High latency (TPOT)	Speculative decoding, quantization, smaller model
Low throughput	Continuous batching, tensor parallelism, larger batch
High cost	Model routing, caching, quantization, smaller model
OOM on GPU	Quantization, smaller batch, gradient checkpointing (training)
Hallucinations	RAG grounding, faithfulness eval, confidence thresholds
Stale knowledge	RAG with real-time indexing, webhook-triggered updates
Traffic spikes	Autoscaling, queueing, model fallback, rate limiting
Multi-tenant	Multi-LoRA serving, per-tenant adapters, isolated caches

System Design Foundations - Session 1

Built through discussion and exercises.

---

1. Functional Requirements

What Are Functional Requirements?

Functional requirements are the **capabilities a system must provide** to solve its intended problem. Without them, the solution is incomplete or fails to fulfill its primary purpose.

Think of it this way: if you remove a functional requirement, does the system still solve the user's core problem? If the answer is **no**, then that requirement is **essential**.

Why They Matter in Interviews

In a system design interview, the interviewer rarely hands you a requirements document. They say something like:

> "Design a URL shortener like bit.ly."

Your first job is to **discover the functional requirements**. If you skip this step and jump straight to hashing algorithms, you miss half the interview.

Discovery Technique: The Removal Test

The fastest way to identify core vs secondary requirements:

> **If removing a capability destroys the product's purpose, it is likely core.**
> **If removing it mainly hurts convenience, it is likely secondary.**

This is not about importance in absolute terms — it is about whether the product **still functions** without it.

Detailed Examples

Gmail

Requirement	Core?	Why?
Send Email	**Yes**	Without this, it is not an email service

Requirement	Core?	Why?
Receive Email	**Yes**	Two-way communication is essential
Store Email	**Yes**	Users expect persistence
Search Email	**Yes**	At scale, browsing is unusable
Spam Filtering	No	Important, but Gmail without spam filtering is still Gmail
Themes / Customization	No	Purely cosmetic

WhatsApp

Requirement	Core?	Why?
Send Message	**Yes**	Core purpose
Receive Message	**Yes**	Core purpose
Store Message	**Yes**	Users expect chat history
Voice Notes	No	Very useful, but not essential to messaging
Status Stories	No	Engagement feature, not core

YouTube

Requirement	Core?	Why?
Upload Video	**Yes**	Without upload, there is no content
Search Video	**Yes**	Discovery is critical at scale
Watch Video	**Yes**	Consumption is the point
Like / Subscribe	No	Engagement features, not core to watching
Comments	No	Community feature, video works without it

Spotify

Requirement	Core?	Why?
Upload Music	**Yes**	Content must come from somewhere
Search Music	**Yes**	Discovery at scale
Listen to Music	**Yes**	Core purpose

Requirement	Core?	Why?
Playlists	No	Important for UX, but not fundamental
Social Sharing	No	Nice to have

Amazon

Requirement	Core?	Why?
List Products	**Yes**	Sellers need to display inventory
Search Products	**Yes**	Discovery at scale
View Products	**Yes**	Users need to see what they are buying
Purchase Products	**Yes**	Core transaction flow
Reviews	No	Trust signal, but Amazon without reviews still sells
Recommendations	No	Conversion driver, not essential

ChatGPT

Requirement	Core?	Why?
Ask Questions	**Yes**	Input mechanism
Get Responses	**Yes**	Output mechanism — this is the product
Conversation History	No	Useful, but single-turn answers still work
File Upload	No	Extended feature

Netflix

Requirement	Core?	Why?
Search Shows	**Yes**	Discovery at scale
Watch Shows	**Yes**	Core purpose
Download for Offline	No	Important for mobile, but not core
Multiple Profiles	No	Family feature

Secondary Requirements Are Still Important

Do not dismiss secondary requirements. In a real interview, after identifying core requirements, you should acknowledge secondary ones and note that they impact **scope and prioritization** but not the fundamental design.

Interview Trap

> "I would use Redis for caching, Kafka for events, and PostgreSQL for the main store."

This is a **trap** because it jumps to implementation before requirements are clear. Always start with:

1. Who is the user?
2. What do they need to do?
3. What happens if they cannot do it?

---

2. Non-Functional Requirements (NFRs)

What Are NFRs?

NFRs describe **how well** the system performs its job. They are the quality attributes of the system.

If functional requirements define **what** the system does, NFRs define **how well** it does it.

The Five Core NFRs

Latency

How fast does the system respond?

- Measured in milliseconds for APIs, seconds for batch jobs.
- **Low latency** means sub-100ms for user-facing requests.
- **High latency** might be acceptable for background processing.
- In ML serving: first-token latency vs time-to-last-token matter differently for different use cases.

Interview question to ask:

> "What is the acceptable p99 latency for this endpoint?"

Availability

Is the system up and usable when users need it?

- Measured as uptime percentage (e.g., 99.9% = ~8.7 hours downtime per year).

- **High availability** systems (99.99%+) require redundancy, failover, and careful design.
- Not every system needs 99.999% availability — it is expensive.

Interview question to ask:

> "What is the business cost of 1 hour of downtime?"

Reliability

Is the system correct and trustworthy?

- Reliability means the system does what it promises, consistently.
- A system can be **available** but **unreliable** (e.g., returning wrong data quickly).
- In ML: reliable means the model outputs are accurate and consistent.

Interview question to ask:

> "What happens if the system returns an incorrect result?"

Scalability

Does the system still work when usage grows?

- **Vertical scaling:** bigger machine (limited).
- **Horizontal scaling:** more machines (the typical cloud approach).
- Scalability is not just about handling more users — it is about handling more **data**, more **requests**, more **geographies**.

Interview question to ask:

> "What is the expected growth in users / data / requests over the next year?"

Security

Can users trust the system with their data?

- Authentication (who are you?), authorization (what can you do?), encryption, audit logging.
- In ML: model inversion attacks, prompt injection, data poisoning.
- Security is often a regulatory requirement (GDPR, SOC2, etc.).

Interview question to ask:

> "What data is sensitive, and who should access it?"

Mental Model for Prioritizing NFRs

Remove the property and ask:

> **What happens to the user?**

This reveals how important the NFR is for **this specific system**.

Detailed Examples

Gmail

NFR	Priority	Reasoning
Latency	Medium	Important, but email is not real-time. Users tolerate 1-2 second send delays
Availability	High	Email is critical infrastructure for many businesses
Reliability	High	Lost emails are unacceptable
Scalability	High	Billions of emails per day
Security	Critical	Email contains sensitive information

WhatsApp

NFR	Priority	Reasoning
Latency	Critical	Messages must feel instant (<200ms ideally)
Availability	Critical	People rely on it for real-time communication
Reliability	Critical	Lost or out-of-order messages break trust
Scalability	Critical	Billions of users, millions of messages per second
Security	Critical	End-to-end encryption is a core promise

ATM

NFR	Priority	Reasoning
Latency	Medium	Users expect quick response, but seconds are acceptable
Availability	Critical	People need cash immediately — failed ATM = angry customer
Reliability	Critical	Wrong balance or failed transaction = legal liability

NFR	Priority	Reasoning
Scalability	Low	Per-ATM load is bounded
Security	Critical	Physical + digital security both matter

Banking

NFR	Priority	Reasoning
Latency	Medium	Transfers can take minutes in many systems
Availability	High	But brief maintenance windows are often acceptable
Reliability	Critical	Incorrect transactions lose money and trust
Scalability	High	Millions of transactions per day
Security	Critical	Regulated industry, fraud prevention is essential

Netflix

NFR	Priority	Reasoning
Latency	High	Buffering is frustrating, but brief pauses are tolerated
Availability	Medium	Short outages are annoying but not catastrophic
Reliability	Medium	Wrong recommendation is not a system failure
Scalability	Critical	Global scale, peak evening traffic
Security	Medium	Account sharing is a bigger issue than data breaches for Netflix

Weather Trading Bot

NFR	Priority	Reasoning
Latency	Critical	Trading opportunities disappear in milliseconds
Availability	High	Downtime = missed trades
Reliability	Critical	Wrong signal = massive financial loss
Scalability	Medium	Bounded by market data volume
Security	High	API keys, trading capital at risk

NFR Tradeoffs

NFRs often conflict:

- **Availability vs Consistency:** CAP theorem territory. Distributed systems must choose.
- **Latency vs Reliability:** More checks = slower but more correct.
- **Security vs Latency:** Encryption and auth add overhead.

In an interview, explicitly discuss tradeoffs. It shows mature system thinking.

Key Lessons Learned

1. Start with the user problem, not the technology.

The worst system design answers start with:

> "I would use Kubernetes and Kafka and Redis..."

The best answers start with:

> "Let me understand who the user is and what they need to do."

2. Separate WHAT from HOW WELL.

- **Functional requirements** = what the system does
- **Non-functional requirements** = how well it does it

Mixing them leads to vague designs. Be explicit.

3. Avoid jumping to implementation details while gathering requirements.

Resist the urge to name databases, queues, or cloud services in the first 5 minutes. Requirements first. Architecture second. Technology third.

4. Use the Removal Test to identify core requirements.

For every capability you list, ask:

> "If I remove this, does the product still serve its core purpose?"

This quickly separates essentials from nice-to-haves.

5. Evaluate NFRs by considering the business impact when they are lost.

NFR prioritization is **context-dependent**. High availability is critical for an ATM but less critical for a weather blog. Always ground NFRs in user impact and business value.

Interview Checklist (Before You Draw a Single Box)

Before touching the whiteboard or mentioning any technology, confirm:

- [ ] Who is the primary user?
- [ ] What are the core functional requirements? (Removal Test)
- [ ] What are the secondary functional requirements?
- [ ] Which NFRs matter most for this system? (Latency? Availability? Reliability?)
- [ ] What are the rough scale estimates? (Users, QPS, data volume)
- [ ] Are there any regulatory or security constraints?

Only after this checklist should you begin drawing the architecture.

L5 Additions: ML-Specific Requirements

ML System Functional Requirements

For ML/LLM systems, functional requirements extend beyond traditional software:

Category	Questions to Ask
Input modality	Text only? Images? Audio? Multi-modal?
Output format	Free text? Structured JSON? Code? Citations?
Knowledge scope	General knowledge? Domain-specific? Real-time data?
Interaction mode	Single-turn? Multi-turn conversation? Streaming?
Personalization	Per-user customization? Memory across sessions?
Tool use	Does the model need to call external APIs/tools?
Safety constraints	What content must be blocked? What must be flagged?

LLM System Examples

Enterprise Document Q&A (RAG)

Requirement	Core?	Why?
Answer questions from documents	**Yes**	Core purpose
Cite sources	**Yes**	Trust — without citations, answers are unverifiable
Handle multi-turn follow-ups	**Yes**	Users need conversational refinement
Support multiple file formats (PDF, DOCX, HTML)	No	Important but extensible
Multi-language support	No	Phase 2 feature
Access control per document	No	Important for enterprise but not core to Q&A

Coding Assistant (Copilot-style)

Requirement	Core?	Why?
Suggest code completions	**Yes**	Core purpose
Context-aware (open files, imports)	**Yes**	Without context, suggestions are useless
Low latency (<200ms)	**Yes**	Must appear as you type
Explain suggestions	No	Useful but not core to completion
Support multiple languages	No	Start with one, expand later
Debug assistance	No	Separate feature

Content Moderation System

Requirement	Core?	Why?
Classify content as safe/unsafe	**Yes**	Core purpose
Handle text + images	**Yes**	Content is multi-modal
Provide reason for flagging	**Yes**	Required for appeals process
Real-time scoring	No	Batch processing acceptable for many use cases
Customizable thresholds	No	Important but configurable post-launch

ML-Specific NFRs

NFR	ML-Specific Concern
Latency	TTFT vs TPOT for LLMs; model inference time vs retrieval time
Availability	Model service downtime vs fallback to cached/simpler model
Reliability	Model correctness — hallucination rate, factual accuracy
Scalability	GPU memory as bottleneck; batch size vs concurrency
Security	Prompt injection, data poisoning, model inversion attacks
Freshness	How often is the model/knowledge base updated?
Cost	Cost per inference; GPU-hours; API costs for external models

L5 Interview Q&A

Q: "What questions would you ask to clarify requirements for an LLM-powered customer support chatbot?"

Functional:
- What channels (web, mobile, email, API)?
- What types of queries (FAQ, troubleshooting, account management)?
- Does it need to escalate to human agents? When?
- Does it need access to user account data?
- Multi-turn conversation or single-turn Q&A?
- What languages?

Non-functional:
- How many concurrent users? Peak QPS?
- What's the acceptable response latency? (TTFT, full response time)
- What's the hallucination tolerance? (Safety-critical vs informational)
- What's the cost budget per query?
- What uptime is required? (24/7 or business hours?)
- What data privacy regulations apply? (GDPR, HIPAA, SOC2?)

ML-specific:
- Does it need to cite sources (RAG)?
- Should it learn from user feedback over time?
- What's the process for updating the knowledge base?
- How do we handle queries the model can't answer?

Q: "How do you determine if a feature is core vs secondary in an ML system?"

Apply the **Removal Test** with an ML twist:

1. **Without this feature, does the model still produce useful output?** If no → core.

2. **Without this feature, is the output trustworthy?** If no → core (safety/grounding).
3. **Without this feature, is the output fast enough to be usable?** If no → core (latency).
4. **Without this feature, is the system cost-effective?** If no → core (cost constraint).

Example: For a medical advice LLM:
- Grounding in medical literature → **core** (without it, hallucinations could harm)
- Citation of sources → **core** (trust requirement)
- Multi-language support → secondary
- Conversational memory → secondary

Q: "What's the biggest mistake candidates make in ML system design interviews?"

Jumping to model architecture before understanding the problem.

Bad answer: "I'd use a 70B parameter LLM with RAG and FAISS for retrieval."

Good answer: "Let me first understand who the users are, what they need, and what constraints we have. Then I'll determine whether we need RAG, what model size is appropriate, and how to serve it."

The interviewer wants to see your **thinking process**, not your knowledge of tools. Requirements → Architecture → Technology. Always in that order.

System Design Foundations - Session 2

Built through discussion and exercises.

---

1. Tradeoffs

The Core Truth of System Design

System design is rarely about finding the perfect solution. It is about **choosing what to optimize** under constraints.

Fast, accurate, reliable, scalable, and cheap **rarely coexist perfectly**.

> **Key Lesson:** Requirements drive tradeoffs.

Why Tradeoffs Are Inevitable

Every engineering decision has a cost:
- Lower latency often means less accuracy (caching, approximation)
- Higher availability often means more complexity (replication, consensus)
- Lower cost often means less performance (shared resources, batching)

Understanding which dimension matters most for **this specific system** is what separates junior from senior engineers.

Real-World Tradeoff Examples

Medical Assistant

Priority	Why
Accuracy dominates latency	Wrong medical advice can cause serious harm
Latency is secondary	A correct slow answer is better than a fast wrong one

Users would rather wait 5 seconds for a correct diagnosis than get an instant hallucination.

Gmail Smart Reply

Priority	Why
Latency dominates	Users see suggestions while typing; they must appear instantly

Priority	Why
Accuracy is secondary	Users can easily ignore or edit bad suggestions

A slightly imperfect but instant suggestion is better than a perfect one that arrives after the user has already moved on.

Coding Assistant (e.g., GitHub Copilot)

This is a nuanced case where the "right" answer depends on context:

- **Low latency** feels magical — suggestions appear as you type
- **Higher accuracy** means less time fixing bad suggestions

The real objective: Developer productivity (time to correct working code)

Sometimes a slightly wrong suggestion that is fast to accept and tweak beats a perfect suggestion that arrives too late.

Interview Framing

When asked to design a system, explicitly state your tradeoffs:

> "For this system, I am prioritizing X over Y because [business reason]. I acknowledge this means [tradeoff consequence]."

This shows structured thinking and business awareness.

---

2. Cost of Error

Not All Mistakes Are Equal

The impact of a wrong answer varies dramatically by domain. Understanding this helps calibrate how much to invest in accuracy, testing, and safety mechanisms.

Error Cost Spectrum

System	Cost of Error	Example
Autocomplete	Low	User sees "thnaks" instead of "thanks" — trivial to correct
Chatbot (general)	Moderate	Wrong restaurant recommendation — annoying but not harmful
Medical Assistant	Very High	Wrong diagnosis recommendation — potentially life-threatening

System	Cost of Error	Example
Weather Trading Bot	Direct Monetary	Wrong weather signal → bad trade → real financial loss
Banking Transaction	Very High	Wrong balance or failed transfer → legal liability
Content Moderation	High	False negative → harmful content spread; false positive → censorship

How Error Cost Drives Design

Low cost of error (autocomplete):
- Can use simpler, faster models
- Less need for guardrails and human review
- Aggressive caching acceptable

High cost of error (medical, banking):
- Need confidence thresholds
- Human-in-the-loop for edge cases
- Extensive testing and validation
- Audit trails for every decision

Interview Sound Bite

> "The cost of error in this domain is [high/moderate/low]. This means I would [specific design choice] to mitigate that risk."

---

3. Success Metrics

The Hierarchy of Metrics

A common mistake is optimizing engineering metrics before understanding business goals.

```
Business Goal
      ↓
Product Metric
      ↓
Engineering Metric
```

Business goals are what leadership cares about.
Product metrics are how PMs measure progress.
Engineering metrics are how engineers optimize.

The danger: engineers often optimize the bottom metric without checking if it moves the top one.

Examples

Weather Trading Bot

Level	Metric
Business Goal	**Profit**
Product Metric	Signal accuracy, trade execution rate
Engineering Metric	Latency, model inference time

Optimizing for sub-100ms inference is useless if the model makes bad predictions. The business goal is profit, not speed.

WhatsApp

Level	Metric
Business Goal	**Adoption / Active Users**
Product Metric	Messages sent, retention rate
Engineering Metric	Message delivery latency, sync speed

If messages are fast but users leave because the app is unreliable, the engineering metric was optimized in isolation.

Netflix Recommendations

Level	Metric
Business Goal	Revenue / Retention
Product Metric	**Watch Time**, completion rate
Engineering Metric	Recommendation latency, model freshness

Watch time is the product metric because it directly predicts retention and subscription value.

Enterprise RAG

Level	Metric
Business Goal	**Reduce support load and save employee time**

Level	Metric
Product Metric	Ticket deflection rate, employee satisfaction
Engineering Metric	Retrieval latency, embedding update frequency

If the RAG system is fast but gives wrong answers, employees still file tickets — the business goal is missed.

Coding Assistant

Level	Metric
Business Goal	Developer productivity, feature velocity
Product Metric	**Developer Productivity** (time to correct working code)
Engineering Metric	Suggestion latency, model accuracy

The real question is not "how fast are suggestions?" but "do developers ship code faster with this tool?"

Anti-Pattern: Optimizing the Wrong Metric

> "Our p99 latency improved from 200ms to 50ms!"

Great. But did user retention improve? Did revenue increase? Did support tickets drop?

Always connect engineering metrics upstream to business impact.

---

4. Key Mental Models

When approaching any system design problem, ask:

The Five Questions

1. **What is the business trying to achieve?**
   - Profit? Adoption? Retention? Cost reduction?
   - This is your north star.
2. **What metric proves success?**
   - Not "what metric is easy to measure?"
   - "What metric, if it improves, proves the business is winning?"
3. **What tradeoff am I making?**
   - Every choice gives up something.
   - State it explicitly.

4. **What happens if the system is wrong?**
   - Cost of error drives safety investment.
   - Medical system vs autocomplete have very different answers.
5. **What happens if the system is slow?**
   - Some systems die from latency (trading bots).
   - Some tolerate it (batch analytics).
   - Know which one you are building.

---

5. Lessons Learned

1. There is rarely a universally correct design choice.

The "best" architecture depends on context. A design that is perfect for WhatsApp might be terrible for a weather trading bot.

2. The correct architecture depends on the objective.

Start with the goal, not the technology. The same building blocks (databases, queues, caches) assembled differently serve different purposes.

3. Business metrics are usually more important than engineering metrics.

A system with 99th percentile latency but no users is worse than a system with 500ms latency and millions of happy users.

4. Strong system designers challenge assumptions and ask what is actually being optimized.

When someone says "we need sub-100ms latency," ask: **why?** What business outcome depends on it? Sometimes the answer reveals the real constraint.

5. User productivity, retention, adoption, and profit often matter more than model accuracy.

In ML systems especially, a 95% accurate model that users love beats a 99% accurate model that is too slow or too fragile to use.

Interview Checklist (Tradeoffs & Metrics)

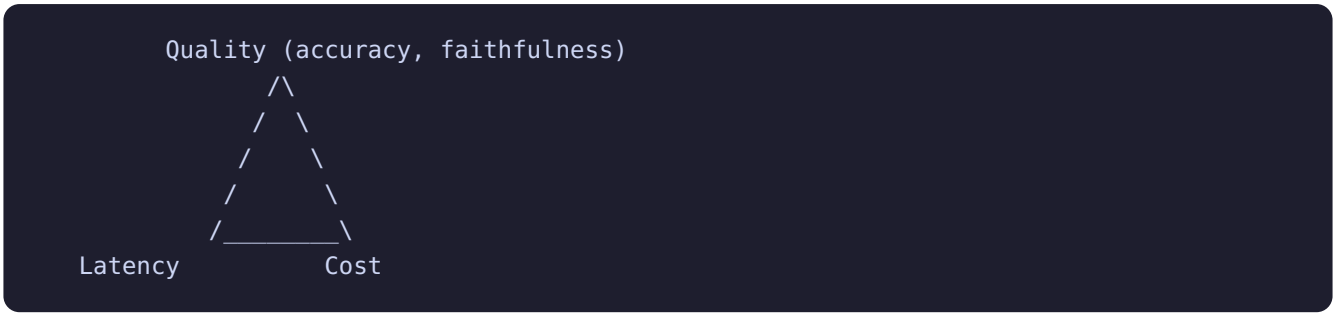
Before proposing any architecture:

- [ ] What is the business goal?
- [ ] What is the product metric that matters?

- [ ] What is the cost of being wrong?
- [ ] What is the cost of being slow?
- [ ] Which NFRs are critical and which are nice-to-have?
- [ ] What tradeoffs am I making, and why?
- [ ] Can I defend this choice if challenged?

L5 Additions: ML-Specific Tradeoffs

The ML Tradeoff Quadrilateral



In ML systems, there's often a **fourth dimension**: **Freshness** (how current is the model's knowledge).

Tradeoff	Example
Quality vs Latency	Larger model = better answers but slower
Quality vs Cost	GPT-4 quality at 100x cost of 8B model
Latency vs Cost	GPU acceleration = faster but expensive
Quality vs Freshness	Fine-tuned model = high quality but stale knowledge
Freshness vs Cost	Real-time RAG indexing = fresh but expensive infra

ML System Tradeoff Examples

Search Ranking System

Priority	Why
Latency dominates	Search results must appear in <100ms
Quality is secondary	Users scroll past imperfect results
Cost is moderate	Can't spend $0.10 per search

Architecture: two-stage retrieval (fast candidate generation → slower reranker on top 50).

Medical Diagnosis LLM

Priority	Why
Quality dominates everything	Wrong diagnosis = patient harm
Latency is secondary	Doctors will wait 30 seconds for a correct answer
Cost is secondary	Healthcare costs are high anyway

Architecture: large model + RAG on medical literature + human-in-the-loop review + confidence thresholds.

Real-time Fraud Detection

Priority	Why
Latency dominates	Transaction must be approved/blocked in <50ms
Quality matters but precision > recall	False positives block legitimate purchases (angry customers)
Cost is moderate	Fraud losses > compute costs

Architecture: gradient-boosted trees (fast inference) + feature store for real-time features + fallback to rules-based system.

Code Generation Assistant

Priority	Why
Latency + Quality both critical	Must be fast AND correct
Cost is secondary	Developer productivity >> API costs
Freshness matters	New APIs, libraries appear constantly

Architecture: medium model (13B-70B) + RAG on documentation + fine-tuned on code + speculative decoding for speed.

Cost of Error in ML Systems

System	False Positive Cost	False Negative Cost	Optimal Strategy
Spam filter	Legitimate email blocked	Spam reaches inbox	Lean toward recall (let some spam through)

System	False Positive Cost	False Negative Cost	Optimal Strategy
Fraud detection	Legitimate transaction blocked	Fraud goes through	Balance — both are expensive
Medical diagnosis	Unnecessary treatment	Missed diagnosis	Lean toward recall (better safe than sorry)
Content moderation	Legitimate content removed	Harmful content stays	Depends on platform values
LLM hallucination	Over-cautious response	False information given	Lean toward precision (refuse rather than hallucinate)

ML-Specific Metrics Hierarchy

```
Business Goal (revenue, retention, cost savings)
      ↓
Product Metric (task completion, user satisfaction, deflection rate)
      ↓
Model Metric (accuracy, faithfulness, hallucination rate, F1)
      ↓
System Metric (latency, throughput, GPU utilization, cost/request)
```

Anti-pattern: "Our model accuracy improved from 92% to 95%!"
Question: Did user retention improve? Did support tickets decrease? If not, the 3% accuracy improvement was irrelevant.

L5 Interview Q&A

Q: "Design a system where both latency and quality are critical."

Example: Coding assistant (Copilot-style)

The key insight is that you need **both** — a slow perfect suggestion is useless (developer has moved on), and a fast wrong suggestion wastes time (developer must debug).

Solution: Cascading models
1. **Fast path**: 3B model generates suggestion in <100ms for 90% of cases.
2. **Slow path**: If the 3B model's confidence is low, use 70B model (<500ms).
3. **Speculative decoding**: 3B drafts, 70B verifies — get 70B quality at 3B speed.

Result: p50 latency = 100ms (3B), p95 = 500ms (70B fallback), quality ≈ 70B model.

Q: "How do you decide between RAG and fine-tuning?"

Factor	RAG	Fine-tuning
Knowledge changes frequently	Real-time updates	Needs retraining
Need citations/sources	Built-in	No source tracking
Domain-specific style/format	Prompt engineering only	Learned behavior
Latency critical	Retrieval adds 50-200ms	Same inference time
Large knowledge base	Scales with vector DB	Limited by training data
Consistent behavior	Depends on retrieval quality	Deterministic

Rule of thumb: RAG for knowledge, fine-tuning for behavior. Use both in production.

Q: "Your LLM system has 95% accuracy but users are unhappy. What do you investigate?"

1. **Are you measuring the right metric?** 95% accuracy on what? If it's accuracy on a benchmark but not on real user queries, the metric is wrong.
2. **What's the 5% failure mode?** If the 5% errors are in the most common query types, users will hit them constantly.
3. **Latency**: Is the system too slow? Users may prefer a faster, slightly less accurate system.
4. **Hallucination rate**: Even 5% hallucination can destroy trust — one confident wrong answer is worse than ten "I don't know" responses.
5. **User expectations**: Are users expecting 100% accuracy? Manage expectations through UI (confidence indicators, citations).
6. **Task completion**: Are users actually completing their tasks? Accuracy ≠ task completion.

Q: "How do you set confidence thresholds for an LLM system?"

```
High confidence (>0.9):  Auto-respond, no human review
Medium confidence (0.5-0.9):  Respond with "I think..." + provide sources
Low confidence (<0.5):  Refuse or escalate to human

Threshold tuning:
  - Start conservative (high threshold) to build trust
  - Monitor false positive rate (confident but wrong)
  - Gradually lower threshold as model improves
  - Different thresholds for different risk levels:
    - Informational query: threshold = 0.3
    - Action-taking query: threshold = 0.8
    - Safety-critical query: threshold = 0.95
```

System Design Foundations - Session 3

Built through discussion and exercises.

---

1. Why Estimate Scale?

Scale Estimation Is Not About Math

The goal is not to produce exact numbers. The goal is to **understand how big the problem is** and whether a simple architecture is sufficient.

What Scale Estimation Reveals

- Can a single machine handle this? Or do we need distributed systems?
- What is the dominant resource constraint? (CPU, memory, disk, network?)
- What will break first as usage grows?
- Is this a problem for a laptop, a server, or a data center?

When Simple Architectures Suffice

If your system serves 1,000 requests per day, you do not need Kubernetes, Kafka, or a microservices architecture. A single server with a database is plenty.

Scale estimation protects you from **over-engineering**.

---

2. Resource Thinking

The Four Key Resources

Every system consumes resources. Understanding which one is the bottleneck drives architectural decisions.

Resource	What It Limits	Typical Bottleneck
Storage	How much data you can keep	Photos, videos, logs, embeddings
Network / Bandwidth	How much data you can move	Streaming, file transfers, APIs
Compute (CPU)	How much processing you can do	Inference, ranking, aggregation
Memory	How much data you can hold in RAM	Caching, model weights, KV cache

WhatsApp Resource Example

For a messaging app, all four matter:

- **Storage:** Billions of messages must be persisted
- **Network:** Every message traverses the network (sender → server → recipient)
- **Compute:** Encryption, protocol handling, push notifications
- **Memory:** Active connections, message queues, recent conversation caches

---

3. Capacity vs Throughput

Two Different Concepts

	Capacity	Throughput
Definition	How much data **exists**	How much work **arrives over time**
Analogy	Size of a lake	Rate of water flowing into it
Unit	Bytes, records, documents	Requests/second, MB/second
Question	"How big is my database?"	"How many queries per second?"

Why the Distinction Matters

- A system with **high capacity** but **low throughput** needs big storage but modest compute (e.g., photo archive).
- A system with **low capacity** but **high throughput** needs fast compute but modest storage (e.g., real-time bidding).
- A system with **both high** needs everything at scale (e.g., WhatsApp, YouTube).

Mental Model: Lake vs Water Flow

- **Lake** = your database (capacity)
- **River feeding the lake** = your incoming requests (throughput)
- A lake can be enormous but fed by a tiny stream.
- A small lake can be fed by a massive waterfall.

Design for whichever is larger or grows faster.

---

4. WhatsApp Estimation Example

Step-by-Step Back-of-the-Envelope

Users

- 1 billion active users

Messages per User

- 100 messages per day per user (average)

Total Messages

```
1B users × 100 messages/day = 100B messages/day
```

Message Size

- Average message: ~100 bytes (text only, excluding metadata)

Daily Storage

```
100B messages × 100 bytes = 10TB/day
```

Annual Storage

```
10TB/day × 365 ≈ 3.6PB/year
```

What This Tells Us

- **Storage:** 3.6PB/year is data-center scale. A single machine cannot hold this.
- **Throughput:** 100B/day ≈ 1.2M messages/second average
- **Compute:** Encryption and routing for 1.2M msgs/sec is significant

Purpose of the Math

The goal is **not** to produce exact numbers. The goal is to understand whether a single machine remains practical.

Conclusion for WhatsApp: No. Distributed systems are required.

---

5. Peak vs Average Traffic

Humans Are Bursty

Average traffic is misleading. Real-world traffic has spikes:

- **Good morning messages:** 7-9 AM local time spikes
- **Office hours:** Sustained high throughput during work hours
- **Sporting events:** Massive spikes during goals, match ends
- **Holidays:** Family group chats explode

WhatsApp Example

Metric	Value
Average throughput	~1.2M messages/second
Peak traffic (assumed 10x)	~12M messages/second

Design Implication

Your system must handle **peak load**, not average load. If you design for average:

- Peak traffic causes cascading failures
- Queues back up
- Latency spikes
- Users see timeouts

Rule of thumb: Design for 3-10x average, depending on burstiness. Know your domain.

---

6. Scale Up vs Scale Out

Two Strategies for Growth

	Scale Up (Vertical)	Scale Out (Horizontal)
What	Bigger machine	More machines
Limit	Hardware ceiling (single box max)	Coordination overhead
Cost	Expensive hardware	Commodity hardware
Complexity	Low (still one box)	High (distributed systems)
When to use	Small scale, temporary	Large scale, permanent

Scale Up

Buy a bigger server:
- More CPU cores
- More RAM
- Faster disk (NVMe, SSD)

Limit: There is always a biggest machine. Eventually you hit the ceiling.

Scale Out

Add more servers:
- Shard data across machines
- Load balance requests
- Replicate for availability

Limit: Coordination becomes expensive. At some point, managing distributed state dominates.

When Distributed Systems Emerge

Distributed systems are not a choice — they are a **consequence** of needing more than one machine can provide.

Once a single machine becomes the bottleneck for **any** resource (CPU, memory, disk, network), you must distribute.

The First Distributed Systems Problem

Once multiple servers exist, you need:

1. **User-to-server mapping (registry):** Which server handles which user?
2. **Server communication:** How do servers talk to each other?
3. **Synchronization:** How do you keep state consistent across servers?
4. **Failure handling:** What happens when a server dies?

These are the foundational challenges of distributed systems.

---

7. Key Lessons

1. Estimation reveals constraints.

You cannot architect without knowing scale. A system for 1K users looks nothing like a system for 1B users.

2. Architecture should emerge from scale.

Do not start with "I will use microservices." Start with "How many users? How much data? What throughput?" and let the architecture follow.

3. Peak traffic matters.

Average traffic is for billing. Peak traffic is for architecture. Design for the spikes, not the calm.

4. Coordination becomes a challenge after scaling out.

The hard part of distributed systems is not running multiple machines — it is making them agree on state, handle failures, and remain consistent.

Interview Checklist (Scale Estimation)

Before proposing architecture:

- [ ] How many users? (DAU, MAU, total)
- [ ] What is the average throughput? (QPS, messages/sec, requests/sec)
- [ ] What is the peak throughput? (3x? 10x?)
- [ ] How much data is generated per user per day?
- [ ] What is the total storage requirement?
- [ ] What is the dominant resource? (CPU, memory, disk, network?)
- [ ] Can a single machine handle this? If not, what must be distributed?
- [ ] What is the read-to-write ratio?

L5 Additions: ML/GPU Scale Estimation

GPU Memory Estimation

```
GPU Memory = Model Weights + KV Cache + Activations + Framework Overhead

Model Weights:
  FP16:  2 bytes × N_params
  INT8:  1 byte  × N_params
  INT4:  0.5 bytes × N_params

KV Cache (per token, per layer):
  2 × n_kv_heads × head_dim × 2 bytes (FP16)

  LLaMA-2 7B:  2 × 32 × 128 × 2 = 16 KB/token/layer × 32 layers = 512 KB/token
  LLaMA-2 70B: 2 × 8 × 128 × 2 = 4 KB/token/layer × 80 layers = 320 KB/token (GQA)
```

LLM Serving Capacity Example

```
Model: LLaMA-2 70B (FP16)
GPU: 4x A100 80GB (tensor parallelism = 4)

Weights:  140 GB / 4 GPUs = 35 GB/GPU
Available for KV cache per GPU: 80 - 35 - 5 (overhead) = 40 GB

KV cache per token (with GQA): 320 KB/token
Max concurrent tokens per GPU: 40 GB / 320 KB = 125,000 tokens
Total across 4 GPUs: 500,000 tokens

If average context = 2048 tokens:
  Max concurrent requests = 500,000 / 2048 ≈ 244 requests

If average response = 200 tokens at 50 tokens/sec:
  Throughput = 244 × 50 = 12,200 tokens/sec
```

Embedding/Vector DB Estimation

```
Documents: 10M documents
Chunk size: 512 tokens → ~20M chunks (avg 2 chunks/doc)
Embedding dim: 1024 (bge-large)
Storage per vector: 1024 × 4 bytes (FP32) = 4 KB

Total vector storage: 20M × 4 KB = 80 GB
HNSW index overhead: ~1.5x → 120 GB
Metadata storage: ~20 GB (Postgres)

Total: ~140 GB — fits on a single machine with 256 GB RAM
```

Training Compute Estimation

```
Training FLOPs ≈ 6 × N_params × N_tokens

Example: Fine-tuning 7B model on 1B tokens
  FLOPs = 6 × 7B × 1B = 4.2 × 10^19 FLOPs

A100 GPU: ~312 TFLOPS (FP16)
  Time = 4.2 × 10^19 / (312 × 10^12) = 134,615 seconds ≈ 37 hours

With 8 GPUs (data parallel): ~4.7 hours
With gradient checkpointing (2x compute): ~9.4 hours
```

Inference Cost Estimation

```
Model: 70B (FP16)
GPU: A100 80GB (cost: ~$2/hour on cloud)

Tokens per second (batch=32): ~2000 tokens/sec
Cost per 1M tokens:
  $2/hour / (2000 × 3600) = $0.0000278 per token
  = $27.80 per 1M tokens

With INT4 quantization (3x throughput):
  = $9.27 per 1M tokens

With model routing (80% to 8B, 20% to 70B):
  8B cost: ~$2.78 per 1M tokens
  Weighted average: 0.8 × 2.78 + 0.2 × 27.80 = $7.78 per 1M tokens
```

L5 Interview Q&A

Q: "Estimate the GPU requirements for serving a 70B model to 1M daily users."

```
1M users × 5 queries/day = 5M queries/day
Average: 500 prompt tokens + 200 response tokens = 700 tokens/query
Total tokens/day: 5M × 700 = 3.5B tokens/day

Peak QPS (5x average): 5M / 86400 × 5 ≈ 290 QPS

70B model on A100 80GB (TP=4):
  Throughput with continuous batching: ~2000 tokens/sec per 4-GPU node
  Nodes needed for peak: 290 QPS × 700 tokens / 2000 = ~102 nodes

  That's 408 A100 GPUs — very expensive.

Optimization:
  - INT4 quantization: 3x throughput → 34 nodes (136 GPUs)
  - Model routing (80% to 8B): 80% handled by 8B (1 GPU each)
    - 8B nodes: 290 × 0.8 × 700 / 4000 = ~41 nodes (41 GPUs)
    - 70B nodes: 290 × 0.2 × 700 / 6000 = ~7 nodes (28 GPUs)
    - Total: 69 GPUs — 6x reduction
```

Q: "How much does it cost to train a 7B model from scratch?"

```
Chinchilla optimal: 7B params × 20 tokens/param = 140B tokens

FLOPs = 6 × 7B × 140B = 5.88 × 10^21 FLOPs

A100 GPU: 312 TFLOPS (FP16, with MFU ~50%): 156 TFLOPS effective
8x A100 node: 1.25 PFLOPS effective

Time = 5.88 × 10^21 / (1.25 × 10^15) = 4.7 × 10^6 seconds ≈ 54 days

Cost: 8 GPUs × $2/hour × 24 × 54 = $20,736

But with LLaMA-style overtraining (2T tokens):
  FLOPs = 6 × 7B × 2T = 8.4 × 10^22
  Time = 8.4 × 10^22 / 1.25 × 10^15 = 67M seconds ≈ 778 days (single node)
  With 64 nodes (512 GPUs): ~12 days
  Cost: 512 × $2 × 24 × 12 = $294,912
```

Q: "How do you estimate vector DB storage for 100M documents?"

```
100M documents → ~300M chunks (avg 3 chunks/doc)
Embedding: 1024 dim, FP32 = 4 KB/vector
Vector storage: 300M × 4 KB = 1.2 TB
HNSW index overhead: 1.5x → 1.8 TB
Metadata (Postgres): ~150 GB

Total: ~2 TB

With INT8 quantized embeddings: 500 GB + 750 GB index = 1.25 TB
With INT4 quantized embeddings: 250 GB + 375 GB index = 625 GB

Shard across 4 machines: 156 GB/machine (INT8) — fits in 256 GB RAM
```

System Design Foundations - Session 4

Distributed Systems Foundations Through Thought Experiments

---

1. Why Replication Exists

The Single-Copy Problem

A single copy of data is risky.

Scenario:
- One copy of your data lives on Server A
- Server A's hard drive dies
- **Your data is gone**

Machines fail regularly. Hard drives fail. Memory corrupts. Power outages happen. Data centers flood.

What Replication Solves

Replication means keeping **multiple copies** of important data so the system can survive failures.

```
One copy  -> Machine dies -> Data may be lost
Multiple copies -> Machine dies -> Data survives elsewhere
```

Replication Is Not Optional at Scale

At small scale, you might get away with backups. At large scale, replication is the only way to ensure:
- **Durability:** Data survives failures
- **Availability:** Users can still access data when some machines are down
- **Read scaling:** Multiple copies can serve read traffic

---

2. Durability vs Reaching a Server

A Critical Distinction

A message **reaching** a server does **not** necessarily mean it is safely stored.

Possible States After a Message Arrives

State	Durability	What Happens on Crash
Message reached server but was never stored	**None**	Message is lost
Stored in memory only	**Low**	Message lost on process restart
Stored on disk (not fsynced)	**Medium**	Message may be lost on OS crash
Stored on disk (fsynced)	**High**	Message survives most crashes
Stored and replicated to multiple servers	**Very High**	Message survives single-server failure

The Key Question

> **When can the system safely say "Success"?**

The answer depends on your durability requirements:
- **Chat app:** Acknowledge after write to disk + replicate
- **Analytics log:** Acknowledge after memory buffer (batch flush later)
- **Banking transaction:** Acknowledge after fsync + multi-server replication + consensus

Interview Trap

> "The message was sent successfully."

Ask: **What does "success" mean?** Stored in memory? On disk? Replicated? The answer reveals the system's durability guarantees.

---

3. Acknowledgements and Tradeoffs

Two Acknowledgement Strategies

Option A: Acknowledge Immediately

```
Client → Server → "OK" (immediately)
         ↓
    (store asynchronously)
```

- **Pro:** Fast response. Low latency.
- **Con:** Risky. If server crashes before storage, the message is lost but the client thinks it succeeded.

Option B: Wait Until Safely Stored

```
Client → Server → Store → Replicate → "OK"
```

- **Pro:** Safe. Message is durable before client is told success.
- **Con:** Slower. Extra disk writes and network round trips.

Domain-Specific Choice

System	Typical Strategy	Why
WhatsApp	Wait for replication	Losing messages breaks trust. An extra 200ms is acceptable.
Analytics logging	Ack immediately	Lost log lines are tolerable. Speed matters more.
Banking	Wait for consensus	Financial correctness is non-negotiable.
Metrics / telemetry	Ack immediately	Brief gaps in monitoring are acceptable.

WhatsApp Example

For WhatsApp, waiting an extra 200ms is **often preferable to losing messages**.

Users notice missing messages. They rarely notice 200ms latency.

---

4. Coordination

The Coordination Challenge

When data exists on multiple servers, the system must coordinate.

Key Coordination Questions

Question	Why It Matters
Did both servers save the data?	If one failed, the user might get inconsistent results
Which copy is authoritative?	If copies diverge, which one is "truth"?
When should success be reported to the user?	Too early = risk. Too late = slow.

Coordination Is Expensive

Every coordination step involves:
- **Network round trips** (latency)
- **Consensus logic** (complexity)
- **Failure handling** (what if the coordinator dies?)

Key principle: Minimize coordination. Coordinate only when necessary.

---

5. The Network Failure Problem

The Fundamental Uncertainty

In distributed systems, if Server A cannot reach Server B, A cannot know with certainty:

- **Did B crash?**
- **Did the network fail?**
- **Did A lose connectivity?**

From A's perspective, all three look identical: **no response**.

Why This Matters

If A assumes B is dead and takes over B's responsibilities:
- What if B is actually alive and also serving requests?
- Now you have **two servers doing the same work** on the same data.
- **Data divergence** is inevitable.

If A waits forever for B:
- A blocks indefinitely.
- The system **stops serving requests**.
- **Availability is sacrificed** for safety.

Distributed Systems Reality

> **Distributed systems often operate with incomplete information.**

This is not a bug. It is a fundamental property of networked systems. All distributed system protocols (consensus, leader election, timeouts) exist to handle this uncertainty.

---

6. Timeouts

Why Timeouts Exist

Because certainty is impossible, systems use timeouts:

```
Wait for some time.
If no response arrives, assume failure and continue.
```

Critical Distinction

> **Assuming failure is not the same as knowing failure.**

When a timeout fires:
- The remote server **might** be dead
- The remote server **might** be slow
- The network **might** be congested
- Your own network card **might** be broken

You **do not know**. You **assume**.

Timeout Tradeoffs

Timeout Duration	Risk
Too short	False positives — healthy servers marked as dead, causing unnecessary failover
Too long	Slow failure detection — users wait, cascading delays
Just right	Detects real failures quickly without excessive false positives

There is no universally "right" timeout. It depends on network characteristics, load patterns, and acceptable failure detection delay.

---

7. Availability vs Consistency

The Network Partition Scenario

Imagine:
- Server A and Server B cannot communicate (network partition)
- Both have copies of the same data
- A user sends a write to A
- Another user sends a read to B

Option: Allow Both to Continue

```
Server A accepts writes → becomes divergent
Server B accepts reads → serves stale data
```

- **Benefit:** High availability. Users can still read and write.
- **Risk:** Different versions of reality emerge. Inconsistency.

The CAP Theorem Connection

When a network partition happens, you must choose:

- **Availability (AP):** Continue serving requests. Risk inconsistency.
- **Consistency (CP):** Reject requests until partition heals. Risk unavailability.

You cannot have both during a partition.

---

8. Split Brain

What Is Split Brain?

Both servers accept changes independently during a partition.

Example:
- Server A stores: `profile_name = "Alice"`
- Server B stores: `profile_name = "Bob"`

When communication returns:

> **Which value is correct?**

This is called a **split-brain situation**.

Why Split Brain Is Dangerous

- **Data loss:** If you pick one value, you lose the other.
- **User confusion:** Different users see different states.
- **Business impact:** Financial records, inventory counts, game scores — all can diverge.

Prevention Strategies

Strategy	How It Works	Tradeoff
Leader-based systems	Only the leader accepts writes. Followers are read-only.	Leader becomes a bottleneck and single point of failure

Strategy	How It Works	Tradeoff
Quorum-based writes	Write must be acknowledged by majority.	Higher latency, more coordination
Conflict resolution	Store both versions, resolve later (CRDTs, vector clocks).	Complex, eventual consistency

---

9. Different Systems, Different Tradeoffs

Domain Determines the Right Choice

Domain	Preferred Approach	Why
Messaging (WhatsApp)	Temporary inconsistency may be acceptable	Users prefer receiving a message eventually over the system being down
Banking	Temporary inconsistency can be catastrophic	Incorrect balances are legally and financially unacceptable
Social media likes	Eventual consistency is fine	A like count that is slightly off is not harmful
E-commerce inventory	Strong consistency preferred	Selling the same item twice = unhappy customers

Key Lesson

> The right tradeoff depends on the **business domain**, not on abstract principles.

What is correct for a chat app is wrong for a bank. Context matters.

---

10. Leaders

The Chain of Command Analogy

Organizations often have chains of command:
- One person makes the final call
- Others follow or advise

Distributed systems use the same idea.

What Is a Leader?

A **leader** is a single authority for important decisions:
- Accepts all writes
- Propagates changes to followers
- Resolves conflicts

Benefits of Leader-Based Design

Benefit	Explanation
Point of truth	One authoritative copy prevents divergence
Reduced conflicts	No split brain — single writer
Clear decision making	Simple mental model for coordination

Common Leader-Based Systems

- **Primary-replica databases:** Primary handles writes, replicas handle reads
- **Raft / Paxos consensus:** Leader coordinates log replication
- **Kafka:** One broker is the leader for each partition

---

11. Leader Failure

What Happens When the Leader Crashes?

Effect	Consequence
New writes may pause	No leader to accept them
Important decisions cannot be made	Consensus stalls
Reads may still continue	Replicas can serve stale reads

The New Question

> **Who becomes the next leader?**

This is the **leader election problem**.

Leader Election Is Hard

- Multiple followers might think they should be leader

- The old leader might not actually be dead (network partition)
- Electing the wrong leader causes split brain

Solutions

Approach	Examples
External coordinator	ZooKeeper, etcd (consensus-based election)
Raft / Paxos	Built-in leader election with safety guarantees
Manual failover	Human decides (slow, error-prone)

---

12. Key Mental Models

Think User First

> **What does the user observe?**

Users do not care about your Raft quorum. They care whether their message sent, their payment went through, their video loaded.

Design from the user experience inward, not from the technology outward.

Think Business First

> **What is the cost of being wrong?**

A wrong movie recommendation costs nothing.
A wrong medical diagnosis costs lives.
A wrong bank transfer costs money and trust.

The architecture must match the business consequence.

Distributed Systems Reality

> - Machines fail.
> - Networks fail.
> - Perfect information does not exist.
> - Most solutions are tradeoffs rather than absolutes.

Accepting these realities is the first step to designing robust systems.

Interview Checklist (Distributed Systems)

When discussing any distributed design:

- [ ] What happens when a single server dies?
- [ ] How is data replicated? How many copies?
- [ ] When does the system acknowledge success? (memory? disk? replicated?)
- [ ] What happens during a network partition?
- [ ] Is the system AP or CP? Why?
- [ ] How is leader failure handled?
- [ ] What is the cost of inconsistency in this domain?
- [ ] What is the timeout strategy?
- [ ] Can I explain this in terms of user impact?

L5 Additions: ML-Specific Distributed Systems

Distributed Training Consistency

Data Parallelism

```
GPU 0: Batch 0 → Forward → Backward → Gradients
GPU 1: Batch 1 → Forward → Backward → Gradients
GPU 2: Batch 2 → Forward → Backward → Gradients
GPU 3: Batch 3 → Forward → Backward → Gradients
                  ↓ All-Reduce ↓
              Averaged Gradients
                    ↓
              Update Weights (all GPUs in sync)
```

Consistency model: Synchronous — all GPUs must finish before weights update. A slow GPU blocks everyone.

Failure handling: If one GPU dies mid-step, the entire step is wasted. Checkpointing every N steps is essential.

Model Parallelism (Tensor Parallel)

```
Weight matrix W (D × D) split across GPUs:
  GPU 0: W[:, 0:D/2]
  GPU 1: W[:, D/2:D]

Forward: x @ W = x @ W_0 || x @ W_1 → all-reduce to combine
```

Consistency: Weights are sharded — no single GPU has the full model. Communication is required for every forward/backward pass.

Pipeline Parallelism

```
GPU 0: Layer 0-19  →  GPU 1: Layer 20-39  →  GPU 2: Layer 40-59  →  GPU 3: Layer 60-79

Micro-batch 1: [GPU0] → [GPU1] → [GPU2] → [GPU3]
Micro-batch 2:       [GPU0] → [GPU1] → [GPU2] → [GPU3]
```

Pipeline bubble: GPU 1 waits for GPU 0, GPU 2 waits for GPU 1, etc. The bubble wastes ~50% of compute with 4 GPUs and 1 micro-batch. Mitigated with many micro-batches.

Model Replication for Inference

```
GPU Group A (4 GPUs): 70B model replica 1
GPU Group B (4 GPUs): 70B model replica 2
GPU Group C (4 GPUs): 70B model replica 3

Load balancer distributes requests across replicas.
Each replica is independent — no coordination needed.
```

Key difference from training: Inference replicas are **stateless** (except KV cache). No consensus needed. If a replica dies, the load balancer routes elsewhere.

KV Cache Consistency

```
Problem: In continuous batching, each request has its own KV cache.
  - No consistency issue (per-request state).

Problem: Prefix caching shares KV cache across requests.
  - If the shared prefix changes (e.g., system prompt update), all cached prefixes are invalidated.
  - Solution: version the system prompt; old cache entries expire.
```

Model Versioning and Rollouts

```
Blue-Green Deployment for Models:
  Blue (current):  v1.0 serving 100% traffic
  Green (new):     v1.1 serving 0% traffic

  1. Deploy v1.1 on separate GPUs (green)
  2. Route 5% traffic to green (canary)
  3. Monitor quality metrics (faithfulness, latency, errors)
  4. If good: ramp to 50% → 100%
  5. If bad: rollback to blue (still running)
```

Shadow deployment: Run v1.1 in parallel (not serving users), compare outputs with v1.0. No user impact, but 2x GPU cost.

Distributed Training Failure Modes

Failure	Impact	Mitigation
GPU dies	Entire step lost	Checkpoint every N steps, resume from last checkpoint
Network partition	Some GPUs can't communicate	Timeout + retry, or skip step
Slow GPU (straggler)	Blocks all GPUs (sync training)	Gradient compression, async training (stale gradients)
OOM on one GPU	Training crashes	Gradient checkpointing, reduce batch size, FSDP sharding
Checkpoint corruption	Hours of progress lost	Redundant checkpoints, checksums

L5 Interview Q&A

Q: "How do you handle a GPU failure during distributed training?"

1. **Checkpointing**: Save model state every N steps (e.g., every 100 steps or 1 hour).
2. **Resume**: Detect failure → load last checkpoint → restart training from there.
3. **Redundant checkpoints**: Store checkpoints on multiple storage systems (NFS + S3).
4. **Fast resume**: Keep optimizer state in checkpoint to avoid re-warmup.
5. **Elastic training**: Some frameworks (TorchElastic) can restart failed workers and rejoin the training group.

Cost of failure: If checkpoint is every 1 hour and GPU dies at 59 minutes, you lose 1 hour of compute. With 512 GPUs at $2/hour, that's $1,024 per failure.

Q: "Synchronous vs asynchronous distributed training — when to use which?"

	Synchronous	Asynchronous
Consistency	All GPUs use same gradients	GPUs use stale gradients from peers
Convergence	Stable, well-understood	Can be unstable (stale gradients)
Throughput	Limited by slowest GPU	No waiting — max throughput
When to use	Most training (preferred)	Very large clusters where stragglers are common

Modern practice: Synchronous with gradient compression (reduce communication) and overlap of compute/communication. Async is rarely used for LLM training because stale gradients cause quality issues.

Q: "How do you roll out a new model version without downtime?"

1. **Blue-green**: Deploy new version on separate GPUs. Switch traffic when ready.
2. **Canary**: Route small % of traffic to new model. Monitor metrics. Ramp up.
3. **Shadow**: Run new model in parallel, compare outputs (no user impact).
4. **Rolling update**: Replace GPU groups one at a time (some downtime risk).
5. **Multi-LoRA**: If only adapter changed, hot-swap LoRA weights (no GPU restart needed).

Monitoring during rollout: faithfulness, hallucination rate, latency, user feedback. Auto-rollback if metrics degrade beyond threshold.

Q: "What's the difference between model replication for inference and data parallelism for training?"

Training (data parallel): All replicas compute gradients → all-reduce to sync → update weights. Requires communication and consensus. Replicas must be in sync.

Inference (model replication): Each replica is independent. No communication needed. Load balancer routes requests. If one replica dies, others continue. Stateless (except per-request KV cache).

The key insight: **training requires consensus, inference does not**. This is why inference scaling is much simpler than training scaling.

System Design Foundations - Session 5

Built through discussion and exercises.

---

1. Single Point of Failure (SPOF)

What Is a SPOF?

A **Single Point of Failure** is one component whose failure can take down the entire system.

```
Client → Load Balancer → [Server A]  ← if this is the only server, it's a SPOF
```

Why SPOFs Are Dangerous

- No redundancy means no fallback
- One hardware failure = total outage
- One network glitch = all users affected

How to Eliminate SPOFs

Strategy	What It Does
Redundancy	Maintain backup instances of critical components
Replication	Multiple copies of data across machines
Failover	Automatic switch to backup when primary fails

Common SPOFs People Forget

- Load balancers (yes, they can be SPOFs too)
- Databases (single primary with no replicas)
- DNS servers
- Identity providers / auth services
- Cloud provider regions (single-region deployment)

---

2. Redundancy

What Is Redundancy?

Maintain **backup instances** of critical components so that if one fails, another takes over.

Types of Redundancy

Type	Example
Active-Active	Two servers both serving traffic. If one dies, the other continues.
Active-Passive	One primary serves, one standby waits. Failover switches to standby.
N+1	N instances needed for load, +1 spare for failures.

Tradeoff

More redundancy = more cost and complexity. But less redundancy = more risk.

Rule of thumb: Redundancy should match the criticality of the component.

---

3. Fault Tolerance

Definition

The system **continues operating** despite failures.

Fault Tolerance vs High Availability

	Fault Tolerance	High Availability
Goal	No downtime at all	Minimize downtime
Cost	Very expensive (full redundancy)	Moderate (some redundancy + fast recovery)
Example	Aircraft flight controls	Web application

Most systems aim for **high availability**, not full fault tolerance. True fault tolerance is expensive.

---

4. Quorum

What Is Quorum?

Majority agreement used for leadership and coordination.

If you have 5 servers, quorum = 3. Any decision agreed upon by 3+ servers is committed.

Why Quorum Works

- Tolerates minority failures (2 of 5 can die)
- Prevents split brain (two majorities cannot coexist)
- Guarantees consistency (majority always overlaps)

Quorum Math

Cluster Size	Quorum	Max Failures Tolerated
3	2	1
5	3	2
7	4	3

Odd numbers are preferred — even-sized clusters tolerate the same failures as one smaller, but with more overhead.

---

5. Randomized Election Timeouts

The Problem

When multiple followers attempt to become leader simultaneously, they can **split the vote** — no one gets a majority.

The Solution

Randomized election timeouts: Each follower waits a random duration before starting an election.

This makes it statistically unlikely that two followers start elections at the same time.

Why It Works

- One follower times out first, starts election
- Others receive the election request and vote
- Split votes become rare

Used In

- **Raft consensus** (core part of the protocol)
- Many leader election systems

---

6. Stable Leadership

Why Stability Matters

Constant re-elections cause:
- Write interruptions
- Cluster instability
- User-visible latency spikes

How to Maintain Stability

- **Longer election timeouts** on stable networks
- **Pre-vote protocol** (Raft optimization): check if others would vote before starting full election
- **Leader lease:** Leader keeps authority for a bounded time even if heartbeats are delayed

---

7. Load Balancer

What It Does

Distributes traffic across multiple servers.

```
         → Server A
Client → Load Balancer → Server B
         → Server C
```

Load Balancing Algorithms

Algorithm	How It Works
Round Robin	Rotate through servers sequentially
Least Connections	Send to server with fewest active connections
IP Hash	Same client always goes to same server (sticky)
Weighted	Distribute based on server capacity

Load Balancer SPOF

Load balancers themselves require redundancy!

If your load balancer dies, all traffic stops — even if all backend servers are healthy.

Solution: Deploy load balancers in active-active or active-passive pairs.

---

8. Cache

What It Does

Store frequently requested data/results for **faster access**.

Cache Hit vs Miss

Event	What Happens
Cache hit	Request served from cache. Fast. No backend call.
Cache miss	Request goes to backend. Result stored in cache for next time.

Cache Invalidation

The challenge of keeping cached data **fresh**.

Strategy	Description
TTL (Time to Live)	Cache entry expires after a set duration
Write-through	Update cache immediately when source data changes
Write-behind	Update cache first, persist to DB later
Explicit invalidation	Manually evict cache entries when data changes

Interview Trap

> "Cache invalidation is one of the hardest problems in computer science."

Know which strategy you are using and **why**.

---

9. Database

What It Does

Persistent storage that survives restarts and power failures.

Common Database Types

Type	Example	Best For
Relational (SQL)	PostgreSQL, MySQL	Structured data, transactions, strong consistency
Document (NoSQL)	MongoDB, DynamoDB	Flexible schemas, horizontal scaling
Key-Value	Redis, Memcached	Fast lookups, caching
Column-family	Cassandra, HBase	Wide-column data, write-heavy workloads
Graph	Neo4j	Relationships, social networks
Time-series	InfluxDB, TimescaleDB	Metrics, monitoring data

---

10. Replication

What It Does

Multiple copies of the same data for availability and read scaling.

Replication Patterns

Pattern	How It Works
Primary-Replica	Primary handles writes, replicas handle reads
Multi-Primary	Any node can accept writes (conflict resolution needed)
Sync Replication	Write acknowledged only after all replicas confirm
Async Replication	Write acknowledged immediately, replicas catch up later

Tradeoff

- **Sync:** Safe but slow
- **Async:** Fast but risk of data loss on failure

---

11. Sharding

What It Does

Split data across machines for storage and write scaling.

Sharding Strategies

Strategy	How It Works
Range-based	Shard by key ranges (e.g., A-M on shard 1, N-Z on shard 2)
Hash-based	Shard by hash of key (even distribution)
Geo-based	Shard by geography (EU users on EU servers)

Challenges

- **Hot shards:** Uneven distribution
- **Cross-shard queries:** Joins across shards are expensive
- **Resharding:** Adding shards requires data migration

---

12. Message Queue

What It Does

Buffers work between producers and consumers.

```
Producer → [Queue] → Consumer
```

Why Queues Help

- **Decouple producers from consumers:** Producers do not wait for consumers
- **Handle traffic spikes:** Queue absorbs burst, consumers process at their own pace
- **Retry on failure:** Failed messages can be reprocessed

Queue Backlogs

When consumers are **slower than producers**, work accumulates in the queue.

Symptoms:
- Queue depth grows
- Latency increases (messages wait longer)
- Eventually, queue may overflow

Solutions:
- Add more consumers (scale out)
- Increase consumer throughput (batching, optimization)
- Backpressure (tell producers to slow down)

---

13. Search Index

What It Does

Precomputed structures for fast search.

Why Not Just Scan?

Scanning every document for a keyword is O(N). An inverted index makes it O(1) lookup + O(matches).

Index Consistency

Indexes must reflect changes in source data.

Problem	Solution
Document updated but index not refreshed	Reindex on write or periodic reindexing
Index and DB out of sync	Eventual consistency with reconciliation

Common Search Systems

- **Elasticsearch:** Full-text search, inverted index
- **Vector DB:** Similarity search, ANN index (HNSW, IVF)

---

14. CDN (Content Delivery Network)

What It Does

Serve content from locations **close to users**.

```
User in India → CDN edge in Mumbai → Content
                    (instead of)
User in India → Origin server in US → Content (slow)
```

Why CDNs Help

- **Lower latency:** Content served from nearby edge
- **Lower origin load:** Edge caches absorb most requests
- **Better availability:** Edge can serve cached content even if origin is down

When to Use a CDN

- Static assets (images, CSS, JS)
- Video streaming
- Large file downloads
- API responses that are cacheable

---

15. Object Storage

What It Does

Store **large files** such as videos, images, and PDFs.

Examples

- **AWS S3**
- **Google Cloud Storage**
- **Azure Blob Storage**

Why Not a Database?

Databases are optimized for structured queries, not large binary blobs. Object storage is:
- Cheaper per GB
- Designed for large files
- Horizontally scalable by default
- Often integrated with CDN for delivery

Cheat Sheet

Problem	Solution
Too much traffic	**Load Balancer**
Repeated expensive work	**Cache**
Persistent data	**Database**

Problem	Solution
Database failure	**Replication**
Database too large	**Sharding**
Traffic spikes	**Queue**
Fast search	**Search Index**
Large files	**Object Storage**
Global latency	**CDN**
Single point of failure	**Redundancy**
Coordination / consensus	**Quorum**
Leader election conflicts	**Randomized Timeouts**

Interview Checklist (Infrastructure Components)

When designing system infrastructure:

- [ ] Where are the SPOFs? How are they eliminated?
- [ ] Is traffic load balanced? How?
- [ ] What is cached? What is the invalidation strategy?
- [ ] What database type? Why?
- [ ] Is data replicated? Sync or async?
- [ ] Is data sharded? What sharding key?
- [ ] Are queues needed for traffic spikes?
- [ ] Is search needed? What index type?
- [ ] Are large files stored in object storage?
- [ ] Is a CDN needed for global users?

L5 Additions: ML/LLM Infrastructure Components

GPU Cluster Architecture

```
Request → Load Balancer → Inference Server (GPU)
                                ↓
                        Model loaded in GPU memory
                                ↓
                        Forward pass → Response
```

GPU-Specific Infrastructure

Component	Purpose	ML-Specific Concern
GPU scheduler	Allocate GPUs to jobs	GPU memory is the bottleneck, not just CPU
Model registry	Store/load model weights	Versioning, A/B deployment, weight distribution
Feature store	Serve ML features in real-time	Low-latency feature retrieval for online inference
Vector DB	Store/retrieve embeddings	HNSW/IVF index, ANN search, metadata filtering
GPU monitoring	Track GPU health	GPU temperature, memory usage, utilization, ECC errors
Checkpoint storage	Save training state	High-throughput write (TBs), distributed filesystem

Vector Database Selection

Vector DB	Type	Best For	Scale
FAISS	In-memory library	Single-machine, low latency	~100M vectors
HNSW (hnswlib)	In-memory graph	High recall, low latency	~50M vectors
Pinecone	Managed service	No-ops, auto-scaling	Billions
Weaviate	Open-source server	Hybrid search (vector + keyword)	~1B vectors
pgvector	Postgres extension	Small scale, SQL integration	~10M vectors
Milvus	Distributed vector DB	Billion-scale, hybrid	Billions
Qdrant	Rust-based, fast	Performance-critical, filtering	~1B vectors

Feature Store Architecture

```
Offline (training):
  Raw data → Feature Engineering → Feature Store (offline)
                                          ↓
Online (inference):
  Request → Feature Store (online, Redis) → Model → Response
```

Consistency: Online and offline features must match. If training uses "user_age_30d" but online serves "user_age_current", the model sees different distributions.

Model Registry

```
Model Registry:
  model_id: "llama-2-70b-chat"
  version: "v1.3"
  weights: s3://models/llama-2-70b-chat/v1.3/weights.bin
  config: s3://models/llama-2-70b-chat/v1.3/config.json
  metrics: {faithfulness: 0.92, latency_p99: 450ms}
  status: "production" | "staging" | "archived"
```

LLM-Specific Caching

Cache Type	What It Caches	Hit Rate	Freshness
Prefix cache	KV cache for shared prompt prefix	High (shared system prompts)	Per session
Semantic cache	Responses for similar queries	20-30%	TTL-based
Exact cache	Responses for identical queries	Low (rare exact repeats)	TTL-based
Embedding cache	Computed embeddings	High (documents don't change often)	On document update

```
def cached_generate(query, cache, model):
    # 1. Check exact cache
    if query in cache:
        return cache[query]

    # 2. Check semantic cache (embedding similarity)
    query_emb = embed(query)
    similar = cache.find_similar(query_emb, threshold=0.95)
    if similar:
        return similar.response

    # 3. Generate new response
    response = model.generate(query)

    # 4. Cache it
    cache.set(query, response, query_emb, ttl=3600)
    return response
```

ML Monitoring Stack

```
System Metrics:
  GPU utilization, memory, temperature, ECC errors
  Network bandwidth (critical for TP/PP)
  Disk I/O (checkpoint loading, model weights)

Model Metrics:
  Inference latency (TTFT, TPOT)
  Output quality (faithfulness, hallucination rate)
  Request rate, batch size, queue depth

Business Metrics:
  User satisfaction (thumbs up/down)
  Task completion rate
  Cost per query
  Revenue impact
```

L5 Interview Q&A

Q: "How do you choose between FAISS, Pinecone, and pgvector for a RAG system?"

Factor	FAISS	Pinecone	pgvector
Scale	<100M vectors	Billions	<10M vectors
Latency	<1ms (in-memory)	~10ms (network)	~5ms (Postgres)
Ops	DIY (you manage)	Managed (no-ops)	Easy (if you have Postgres)

Factor	FAISS	Pinecone	pgvector
Cost	Free (self-hosted)	$$ (managed)	Free (if you have Postgres)
Filtering	Limited	Rich metadata filtering	Full SQL filtering
Best for	Prototype, low-latency	Production at scale	Small scale, SQL integration

Decision: Start with FAISS for prototyping → move to Pinecone/Milvus for production at scale → use pgvector if you already have Postgres and scale is small.

Q: "Design a feature store for a real-time ML system."

```
Offline path (batch):
  Data warehouse → Spark/Beam feature pipeline → Offline feature store (S3/BigQuery)
  Used for: training data generation, batch scoring

Online path (real-time):
  User request → Online feature store (Redis/DynamoDB) → Model inference
  Used for: real-time scoring (<10ms retrieval)

Sync: Offline → Online sync pipeline (hourly or streaming)
```

Key challenges:
1. **Online/offline consistency**: same feature definition, same transformations.
2. **Freshness**: how quickly do new features appear online? (streaming vs batch)
3. **Point-in-time correctness**: training must use features as they were at event time, not current values.

Q: "How do you implement semantic caching for an LLM serving system?"

```
1. Compute embedding of incoming query
2. Search cache for similar queries (cosine similarity > 0.95)
3. If found: return cached response (with TTL check)
4. If not: generate response, cache with embedding + TTL

Cache structure:
  Key: query embedding (1024-dim vector)
  Value: response text + timestamp + metadata
  Index: HNSW for fast similarity search
  TTL: 1 hour (or invalidate on knowledge base update)

Hit rate: typically 20-30% for production LLM systems
Savings: 20-30% fewer inference calls → significant cost reduction
```

Risks:
- **Stale answers**: cached response may be outdated. Mitigate with TTL.
- **False positives**: semantically similar but functionally different queries. Mitigate with high threshold

(0.95+).
- **Cache invalidation**: when knowledge base updates, invalidate relevant cache entries.

Q: "What infrastructure do you need to serve 10 different LLM models?"

```
Model fleet:
  - 2x large models (70B): 8 GPUs each (TP=4, 2 replicas)
  - 3x medium models (13B): 2 GPUs each (TP=1, 2 replicas each)
  - 5x small models (8B): 1 GPU each (2 replicas each)

Total GPUs: 16 + 12 + 10 = 38 GPUs

Components:
  - Model registry: track versions, weights, configs
  - GPU scheduler: allocate GPUs to models, handle failures
  - Load balancer: route to correct model + replica
  - Health checker: GPU temperature, memory, inference latency
  - Autoscaler: add/remove replicas based on load
  - Monitoring: per-model metrics (QPS, latency, quality)
  - Shared cache: prefix cache for common system prompts
```

Q: "How do you handle GPU OOM in production?"

1. **Prevention**:
   - Set max batch size based on available memory.
   - Set max context length per request.
   - Monitor memory usage, alert at 80%.
2. **Detection**:
   - Catch OOM errors from inference engine.
   - Log the request that caused OOM (prompt length, batch size).
3. **Recovery**:
   - Drop the offending request, return error to user.
   - Reduce batch size temporarily.
   - If persistent: restart the inference server (reload model).
4. **Long-term**:
   - Quantize model (FP16 → INT8 → INT4).
   - Use GQA/MQA to reduce KV cache memory.
   - Add more GPUs (increase TP or add replicas).
   - Implement request admission control (reject very long prompts).